Preguntas de examen PSyD

Daniel Báscones García

3 de noviembre de 2026

Índice

1. Introducción y aviso **1**

2. Preguntas **1**

1. Introducción y aviso

En las siguientes páginas, se muestran **ejemplos** de preguntas de examen. Esto no quiere decir, ni que todas las preguntas vayan a salir de aquí (seguramente no), ni que sean refritos o adaptaciones de las mismas. Los contenidos que entran en examen incluyen tanto la teoría de los documentos teóricos, como la de las prácticas, y puede entrar cualquier cosa de aquellos siempre y cuando se haya dado en clase.

Así pues, esta colección sirve solo como referencia para entender la profundidad y dificultad que se puede esperar del examen, pero en ningún caso sustituye su resolución a estudiar la materia.

2. Preguntas

1. Complete la siguiente función:

```c
/**
 * @param value es un puntero al dato que se cambia
 * @param byte_index es un índice (0-3) que indica el byte a cambiar
 * @param byte_value es el valor que hay que cambiar
 * @example Por ejemplo, si *value=0xabababab, byte_index=2,
 *          y byte_value=0xcd, el resultado será, que tras la llamada,
 *          *value=0xabcdabab
 */
static inline void set_byte(uint32_t * value, uint32_t byte_index, uint8_t
    byte_value) {

}
```

2. El periférico `CRC` (ver Figura 1) sirve para generar códigos de comprobación que verifican la integridad de los datos. Su acceso se realiza a partir de la dirección `0x40023000`. Escriba el código C necesario como para inicializar el registro `CR` sin cambio de orden de datos (bits `REVx` a cero), ancho de 8 bits (`10b` en `POLYSIZE`) y forzando un reseteo (`RESET` a 1, este bit se limpia solo tras activarlo). Tenga en cuenta que no se permite un acceso

Table 39. CRC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	CRC_DR	DR[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x04	CRC_IDR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	IDR[7:0]							
	Reset value																									0	0	0	0	0	0	0	0
0x08	CRC_CR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	REV_OUT	REV_IN[1:0]		POLYSIZE[1:0]		Res	Res	RESET
	Reset value																									0	0	0	0	0			0
0x10	CRC_INIT	CRC_INIT[31:0]																															
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
0x14	CRC_POL	POL[31:0]																															
	Reset value	0	0	0	0	0	1	0	0	1	1	0	0	0	0	0	1	0	0	0	1	1	1	0	1	1	0	1	1	0	1	1	1

Figura 1: Tabla de registros del periférico CRC

directo a la dirección de memoria, sino que hay que crear una estructura de datos para el periférico.

3. ¿Qué efecto produce en la compilación calificar un puntero a un registro con la palabra clave `volatile` en lenguaje C?

4. Escriba una función en C `void gpio_set_mode(GPIO_TypeDef *GPIOx, uint32_t pin, uint32_t mode)` que configure de forma atómica a nivel de software el modo de un pin en el registro `MODER` (donde cada pin ocupa 2 bits). Asegúrese de limpiar primero la configuración previa mediante máscaras binarias antes de aplicar el nuevo modo

5. En el microcontrolador STM32F723, el controlador de interrupciones NVIC maneja registros vectorizados como `ISER` (Interrupt Set Enable Register), definidos dentro del `struct` del periférico como un array `volatile uint32_t ISER[8]` para cubrir hasta 256 líneas de interrupción. Explique matemáticamente cómo se traduce un número global de línea de interrupción `IRQn` (por ejemplo, 71) a un índice dentro del array `ISER[]` y a una posición de bit dentro de ese elemento de 32 bits.

6. Dado el siguiente *linker script*:

```
MEMORY
{
    RAM (xrw)    : ORIGIN = 0x20000000, LENGTH = 256K
    FLASH (rx)   : ORIGIN = 0x08000000, LENGTH = 512K
}

SECTIONS
{
    .isr_vector : {
        KEEP(*(.isr_vector))
    } >FLASH

    .text : {
        *(.text*)
        *(.rodata*)
```

```
    } >FLASH

    .rodata :
    {
        *(.rodata)
        *(.rodata*)
    } >FLASH

    _sidata = LOADADDR(.data);

    .data :
    {
        _sdata = .;         /* Inicio de datos en RAM */
        *(.data)            /* .data sections */
        *(.data*)           /* .data* sections */
        _edata = .;         /* Fin de datos en RAM*/

    } >RAM AT> FLASH

    .bss : {
        _sbss = .;
        *(.bss*)
        _ebss = .;
    } > RAM

    _estack = ORIGIN(RAM) + LENGTH(RAM);
}
```

- Realice un esquema del mapa de memoria de las diferentes secciones.
- Indique quién es el responsable de inicializar la memoria RAM, dónde reside su código, y cómo se llega a ejecutar.

7. Compare el funcionamiento del modelo de E/S por encuesta (*polling*) frente a las interrupciones. Analiza las ventajas y desventajas de cada uno en relación con la eficiencia de la CPU, latencia de respuesta y complejidad del código.

8. ¿Qué datos contiene la tabla de vectores de un procesador? ¿En qué contexto y cómo se utilizan?

9. ¿Qué tareas básicas ha de realizar la rutina de inicialización `Reset_Handler` antes de saltar a `main`?

10. A la hora de configurar las rutinas de tratamiento de interrupción (ISRs), se utilizan los siguientes fragmentos de código:

```
//1)
    . section . isr_vector , " a ", % progbits
    . type g_pfnVectors , %object

    g_pfnVectors :
        . word _estack /* 0 x00 : Pila inicial */
        //...
        . word USART1_IRQHandler
        //...

//2)
    . weak USART1_IRQHandler
    . thumb_set USART1_IRQHandler , Default_Handler

//3)
```

```
    void USART1_IRQHandler ( void ) {
        //...
    }
```

Indica qué función cumple cada uno de ellos.

11. Indica, preferiblemente con un diagrama, el flujo de ejecución cuando una ISR interrumpe el hilo principal de ejecución. En particular, señala qué ocurre con los registros y la tabla de vectores del procesador.

12. Explica las diferencias entre los `Callbacks` frente a gestionar ISRs directamente.

13. Explique por qué en microcontroladores modernos la señal de reloj de cada periférico se encuentra desactivada por defecto tras un reinicio. ¿Cómo se gestiona su activación, y qué ocurre si se intenta acceder a los registros de un periférico cuyo reloj no ha sido habilitado previamente?

14. ¿Qué es un *Watchdog* y qué función tiene dentro de un procesador?

15. ¿Qué sucede al intentar modificar los registros de control de un periférico (por ejemplo `GPIOA`) si no se ha activado previamente su reloj en el registro `RCC` correspondiente? (En este caso, `RCC_AHB1ENR`)

16. Si la frecuencia de reloj de entrada del periférico `USART6` es de 16 MHz y se requiere una velocidad de 115200 baudios ¿qué valor (aproximadamente) debe cargarse en el registro `USART6_BRR`?

17. Explica cómo se consiguen transmitir datos por una `UART`. ¿Cómo se sincronizan emisor y receptor?

18. En un `GPIO`, los pines se pueden configurar en varios modos: entrada, salida, modo alternativo y modo analógico. ¿Qué funcionalidad nos ofrece el modo alternativo, y cómo se utiliza?

19. El gestor de interrupciones externas `EXTI` tiene sólo unas pocas líneas (en concreto 16), muchas menos que la totalidad de pines de los que dispone el procesador (16 pines por puerto, en 10 puertos diferentes). ¿Cuántos pines puedo utilizar simultáneamente para interrumpir al procesador? ¿Qué restricciones existen?

20. Asumiendo una frecuencia de entrada de 16MHz, calcule los valores de los registros de *preescalado* y *auto-reload* para conseguir un timer que se dispare cada 10 milisegundos.

21. Defina las variables y funciones necesarias para implementar un *ticker* de sistema que mida el tiempo en milisegundos mediante llamadas a una función `GetTick`. Asuma que el periférico ya está configurado para generar las interrupciones, y que la ISR que lo gestiona se llama `Systick_Handler`.

```
volatile uint32_t msTicks;

static uint32_t inline GetTick (void) {
    return msTicks;
}

void SysTick_Handler (void) {
    msTicks++;
}
```

22. Compara las fuentes de reloj internas (HSI, LSI) con las externas por cristal de cuarzo (HSE, LSE) en términos de precisión, y consumo energético. ¿Por qué utilizaríamos un oscilador de cuarzo de 32,768 kHz para el reloj en tiempo real (RTC)?

23. Describe la arquitectura física del bus I2C (líneas y funcionamiento). Explica la secuencia de pasos (a nivel lógico) que debe seguir un maestro para realizar una operación de lectura de un registro en un dispositivo esclavo.

24. En el control de periféricos mediante el bus I2C, el periférico no puede iniciar una transacción. Explique en este contexto si se pueden, y cómo, tratar las interrupciones.

25. Explique las principales diferencias entre I2C y SPI, en cuanto a líneas utilizadas, direccionamiento de periféricos, e interrupciones.

26. Explica por qué es obligatorio borrar antes de escribir un dato en memoria `FLASH`.

27. Al almacenar datos de configuración en memoria no volátil, ¿cuál es el propósito de incluir un patrón o *magic number*? Describe el flujo lógico durante el arranque para decidir si los datos leídos son válidos o deben cargarse los valores por defecto.

28. Explica la diferencia entre un planificador multitarea cooperativo y uno expropiativo. Analiza qué ocurre en un sistema cooperativo si una de las tareas entra en un bucle prolongado o bloqueante.

29. En un RTOS, explica la relación de las interrupciones `PENDSV` y `SysTick` con el cambio de tarea.

30. Explique, en el contexto de un RTOS, las diferentes pilas o *Stacks* que existen, así como sus funciones y contenidos.